Green Coding and Digital Carbon Footprint Optimisation Tool

Sakarya University · COP31 project of the Social Contribution and Sustainability Coordination Office
Team: Yunus Emre Sevindik · Mehmet Zahid Görgeç · Version: October 2026 · Open source: github.com/yes3760/yesil-kod

SUMMARY

Yeşil Kod ("Green Code") **finds** energy-wasting patterns in Python code, **rewrites** the code itself, **verifies** that both versions return exactly the same result, and **measures** the energy used for the same work, converting it to gCO_2e with the carbon intensity of the Türkiye grid. In all four realistic scenarios the fix was produced by the engine; CPU energy fell by 2.4 to 4,049 times and the outputs did not change.

465×
from fixing a single line, same result

01 Problem

Data-centre electricity use is expected to grow from **415 TWh** in 2024 to **~945 TWh** in 2030 (IEA, 2025). Debates about efficiency usually focus on hardware, while the software itself goes unnoticed: a one-line waste inside a loop becomes a real electricity and carbon load when it runs millions of times a day. Developers cannot see this load, because the energy of a line of code is measured and shown nowhere.

02 Solution: four steps

1 Find

Without running the code, its syntax tree (AST) is scanned and nine waste patterns are flagged line by line.

2 Fix

The engine rewrites four patterns and touches only the changed lines; comments and formatting are kept.

3 Verify

Original and fixed code run on the same data; if the output is not identical, the fix is rejected.

4 Measure

Both versions handle the same number of requests; CPU energy is measured and converted to carbon.

RULE	WASTE PATTERN	SEVERITY	ENGINE'S FIX
YK001	Database query inside a loop (N+1)	high	One batched IN query before the loop
YK002	List lookup inside a loop	high	set() built before the loop
YK003	Repeating loop-invariant work	medium	Call moved out of the loop
YK004	Uncached double recursion	high	functools.lru_cache added
YK005	Busy polling	medium	Advice: event-driven design
YK006	Nested loop, $O(n^2)$	info	Advice: single pass with a dict
YK007	Iterating a pandas table row by row (iterrows, apply axis=1)	high	Advice: vectorised pandas/numpy
YK008	HTTP request without a session in a loop	medium	Advice: reuse the connection with requests.Session
YK009	Adding or removing at the front of a list	medium	Advice: collections.deque

03 How it is used

- **Live panel:** fix, equivalence test and energy race on the stand screen.
- **Command line:** analysis, fixing, measurement and reports.
- **Continuous integration:** a report on every pull request; the check stops on high-severity findings.
- **Web:** visitors analyse their own code and measure the scenarios on their phone; the code never leaves the device.

04 Technical design

- Python standard library only; no installation or internet needed.
- Intel/AMD **RAPL** counters on Linux, Apple's CPU power data (**powermetrics**) on Mac; CPU time × per-core power without permission.
- 20 unit tests, run automatically on Python 3.9 and 3.13 with GitHub Actions on every change.

05 Results

Each scenario represents a waste that is common in real projects. None of the fixed code was written by hand; the engine produced all of it and every version passed the equivalence test.

SCENARIO	WASTE → ENGINE'S FIX	IN REAL LIFE	ENERGY REDUCTION
Route count	uncached double recursion → lru_cache	route, pricing and recommendation maths	**3,640–4,049×**
Attendance check	4,500 IDs searched in a 6,000-item list → set()	attendance, stock and permission checks	**456–698×**
Price list	config and regex rebuilt for every product → moved out of the loop	APIs that read config on every request	**51–55×**
Order report	a separate customer query for each of 4,000 orders → batched IN	e-commerce lists, dashboards	**2.4–2.9×**

Reduction in CPU energy per request; four separate runs on the same computer (Apple M4) on 26 October 2026, estimated mode. Each run is the mean of 5 repeats. In the order report, database queries drop from 4,001 to 2; the database runs on the same machine, so network savings are not included in this ratio. At one million requests a day, the attendance line emitted about 41–89 kg CO_2e a year.

06 Measurement method

- **Carbon:** SCI = (E × I + M) / R. E measured energy, I Türkiye grid 341 gCO_2e/kWh (Aug 2025–Jul 2026), R per request.
- **Fairness:** short code is repeated until the counters can measure it; every result is the mean of 5 repeats.
- **Caching:** caches in the fixed code are cleared before every request, so gains never come from earlier runs.
- **Idle power:** in hardware mode the machine's idle power is measured and subtracted from every result.

07 Honest limits

- Only CPU energy is measured; memory, network, disk and cooling are excluded. The gain shown is a lower bound.
- In estimated mode, ratios are more reliable than absolute values; ratios vary between machines.
- The engine currently supports Python; it fixes four of the nine patterns automatically and advises on the rest.
- Grid carbon intensity is an annual average; the grid changes hour by hour.

08 Roadmap

- **More languages:** the same four-step design for JavaScript/TypeScript and Java.
- **Full SCI:** include embodied hardware emissions (M) and memory energy.
- **Carbon-aware scheduling:** shift heavy jobs to cleaner hours using hourly generation data from EPİAŞ (Türkiye's energy exchange).
- **Developer tools:** an editor extension and a ready-made CI step.
- **Pilot:** scan student software projects at the university for waste and report the measured savings.

09 Sustainable Development Goals

7 Affordable and clean energy: the same work with less electricity

9 Industry, innovation and infrastructure: efficient digital infrastructure

12 Responsible consumption and production: not wasting computing resources

13 Climate action: making the carbon of software measurable